



Amazon Aurora

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Introduction to Aurora



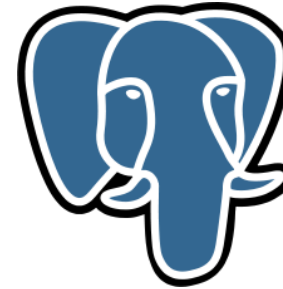
Amazon Aurora is a fully managed relational database cluster that

- combines the **speed** and **availability** of **high-end databases**
- with the **simplicity** and **cost-effectiveness** of **open-source databases**.

Aurora can run either MySQL-compatible or Postgres-compatible engines



Aurora MySQL is **5x better performance** than traditional MySQL



Aurora Postgres is **3x better performance** than traditional Postgres

1/10th the costs of other solutions offering similar performance and availability.



Aurora contains most of the other features RDS and has its own exclusive features

Aurora - Scaling

Durability and Fault Tolerance

- Aurora Backup and Failover are handled **automatically**
- Snapshots of data can be **shared** with other AWS accounts



Storage is **self-healing**, in that data blocks and disks are continuously scanned for errors and repaired automatically.

Availability

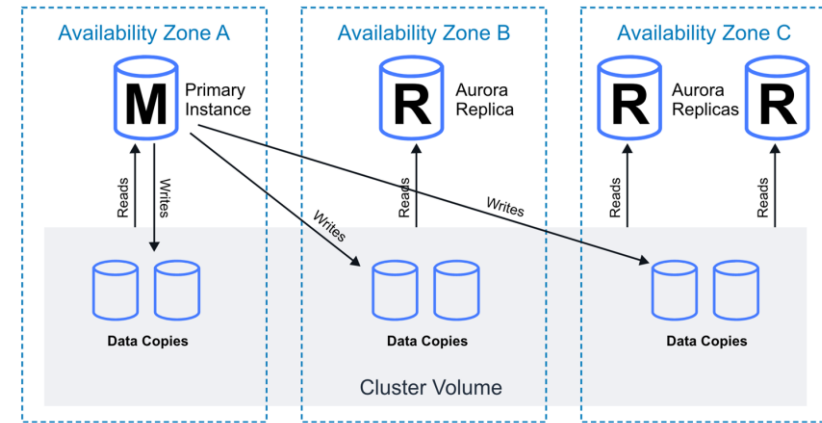
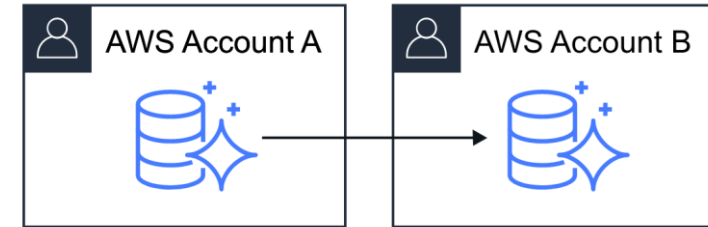
- Aurora deploys in a minimum of **3 availability zones** each **contain 2 copies of your data at all times**.
- That means there are **6 copies**
- Lose up to 2 copies of your data without affecting **write** availability.
- Lose up to 3 copies of your data without affecting **read** availability.

Storage

- A cluster starts with 10GB of storage and scale in **10GB** increments up to 64TB or 128 TB depending on DB engine version.
- Storage is autoscaling.
- Computing resources can scale up to **32 vCPUs** and **244GB** of memory.

Security

- TLS/SSL certificate can be applied to encrypt security connections so termination occurs at the database
- Data is encrypted-at-rest by default and cannot be turned off. Can use KMS keys



Aurora Provisioned

Aurora Serverless Provisioned is the default compute configuration for Aurora.

Aurora DB cluster contains a primary DB instance that performs reads and writes, and, optionally, up to 15 Aurora Replicas (reader DB instances).

Creating an Aurora DB provisioned cluster via the AWS CLI

```
aws rds create-db-cluster \  
--db-cluster-identifier myauroracluster \  
--engine aurora-mysql \  
--engine-version 5.7.12 \  
--master-username masteruser \  
--master-user-password masterpassword \  
--backup-retention-period 7 \  
--preferred-backup-window 07:00-09:00 \  
--preferred-maintenance-window Sun:23:00-Mon:01:00
```



The primary DB instance will not be created for by default you have to create your instances after creating your cluster

Aurora Reader and Writer Instances

Amazon Aurora have two types of instances within a cluster: Writers and Readers.

	Writer Instance	Reader Instance
Role	Handles writes; can also read	Handles reads
Quantity	One per cluster	*15 per cluster for scalability
Scalability	Vertical only (upgrade instance)	Horizontal (add more instances)
Availability	Critical; failure triggers failover	Distributes reads; can be failover target
Use Cases	Transactional changes	Read-heavy workloads, analytics
Failover Capability	Automatic promotion of a Reader if failed	Can be promoted to Writer
Cost	Based on instance size and IOPS	Increases with each instance added

Reader Instances are also known as **Aurora replicas**

Aurora Reader and Writer Instances

The **first DB instance** you create in your cluster will be writer instance.

```
aws rds create-db-instance \  
--db-instance-identifier my-writer-instance \  
--db-instance-class db.r4.large \  
--engine aurora-mysql \  
--db-cluster-identifier my-cluster
```



All **other instances created afterwards** will be a reader instance (Aurora replica)

```
aws rds create-db-instance \  
--db-instance-identifier my-reader-instance \  
--db-instance-class db.r4.large \  
--engine aurora-mysql \  
--db-cluster-identifier my-cluster
```



Aurora Serverless V2

Aurora Serverless V2 **fully-manages the autoscaling configuration** for Amazon Aurora

- Capacity is adjusted automatically based on application demand
- You're charged only for the resources that your DB clusters consume
- suitable for the most demanding, highly variable workloads
- Aurora “Serverless” V2 **does not scale to zero** and must maintain at least 0.5 ACUs
- Only certain Aurora Instances classes are available to use with Aurora Serverless V2

Aurora capacity unit (ACU)

ACU's is the unit of measurement Aurora Serverless V2 uses to determine cost vs capacity.

1 ACU is about 2GiB of memory, CPU and networking.

- Capacity ranges between 0.5 ACUs and 128 ACUs



As of December 31, 2024, Amazon Aurora will no longer support Serverless V1

Aurora Serverless V2

When you configure your Aurora Cluster for Serverless V2
you **set a Minimum and Maximum ACU capacity**

```
aws rds create-db-cluster \  
--db-cluster-identifier my-cluster \  
--region us-east-1 \  
--engine aurora-mysql \  
--engine-version 8.0.mysql_aurora.3.04.1 \  
--serverless-v2-scaling-configuration MinCapacity=1,MaxCapacity=4 \  
--master-username myuser \  
--manage-master-user-password
```



When you create a Writer instance you need to specify the
db class as being **db.serverless**

```
aws rds create-db-instance \  
--db-cluster-identifier my-serverless-v2-cluster \  
--db-instance-identifier my-serverless-v2-instance \  
--db-instance-class db.serverless \  
--engine aurora-mysql
```



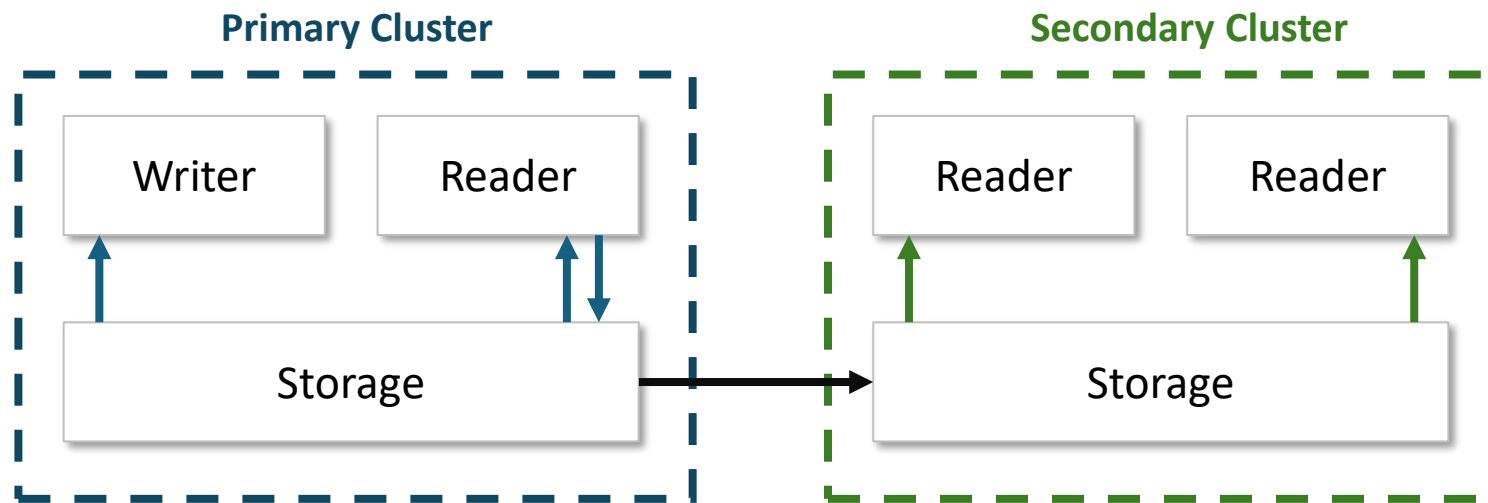
Aurora Serverless V2 vs Provisioned

	Aurora Serverless V2	Aurora Provisioned
Scaling	Fine-grained, almost instant scaling.	Manual scaling; requires planning and downtime.
Capacity Range	0.5-128 ACUs, more flexible.	Fixed, based on instance size chosen.
Scaling Speed	Seconds	N/A (manual intervention required).
Read/Write Scaling	Independently	Depends on instance type and read replica configuration.
Compatibility	Broader version support.	Wide version support, depending on instance type.
Use Cases	Highly variable workloads needing immediate scaling.	Stable workloads with predictable performance needs.
Billing	ACUs per second, more granular, + storage.	Instance hours + storage.
Start/Stop	Responsive start/stop, cost-saving for intermittent loads.	Manual start/stop.
Maintenance	Minimal downtime, more seamless.	Scheduled maintenance windows.

Aurora Global Database

Amazon Aurora Global Database is a **Aurora database spanning multiple regions** for **global low-latency** and high availability.

- Has a primary cluster in 1 region
- Has up-to 5 secondary DB clusters in different regions
- Write operation occur on the Primary cluster
- Data is replicated to secondary cluster (typically under a second)
- Global Database is only available in specific regions and specific database versions



Aurora Global Database

You need to create a **global cluster**

```
aws rds create-global-cluster \  
--region primary_region \  
--global-cluster-identifier global_database_id \  
--engine aurora-mysql \  
--engine-version version # optional
```

Then you **create your primary cluster** and **use the global cluster identifier** to place it within the global cluster

```
aws rds create-db-cluster \  
--region primary_region \  
--db-cluster-identifier primary_db_cluster_id \  
--master-username userid \  
--master-user-password password \  
--engine aurora-mysql \  
--engine-version version \  
--global-cluster-identifier global_database_id
```

Not shown, creating the db instance on the cluster.

Then you **create your secondary cluster** and **use the global cluster identifier** to place it within the global cluster

```
aws rds create-db-cluster \  
--region secondary_region \  
--db-cluster-identifier secondary_cluster_id \  
--global-cluster-identifier global_database_id \  
--engine aurora-mysql \  
--engine-version version
```

Not shown, creating the db instance on the cluster.

Aurora – RDS Data API

RDS Data API allows you to use **HTTP requests to securely query** an Aurora database.

- Unlimited max request per seconds
- Must be enable on the cluster to user
- DATA API calls are by default excluded by CloudTrail since they are data events
- Multi statements aren't supported
- Cannot retrieve multi-dimensional arrays for a query's column
- Supports specific data-types
- Supports Execution and Transaction statements.

Enabling RDS DATA API on an Aurora Cluster

```
aws rds enable-http-endpoint \  
--resource-arn cluster_arn
```

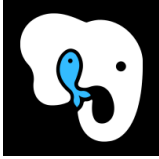
Executing an SQL statement using the RDS Data API

```
aws rds-data execute-statement \  
--resource-arn "arn:aws:rds:us-east-1:123456789012:cluster:mydbcluster" \  
--database "mydb" \  
--secret-arn "arn:aws:secretsmanager:us-east-1:123456789012:secret:mysecret" \  
--sql "select * from mytable"
```



Aurora in the AWS Management Console has a **Query Editor** which is just an interface to connect and use the RDS DATA API.

Aurora – Babelfish for Aurora



What is Babelfish?

Babelfish for PostgreSQL is an open source library with the capability for PostgreSQL to understand queries from applications written for Microsoft SQL Server.

Babelfish for Aurora PostgreSQL extends your Aurora PostgreSQL DB cluster with the ability to accept database connections from Microsoft SQL



Apps originally built for SQL Server can work directly with Aurora PostgreSQL with few code changes compared to a traditional migration and without changing database drivers.

Both SQL dialects supported by Babelfish are available through their native wire protocols at the following ports:

- SQL Server dialect (T-SQL), port 1433.
- PostgreSQL dialect (PL/pgSQL), port 5432.
- Babelfish runs the Transact-SQL (T-SQL)
- Babelfish currently doesn't support
 - RDS Blue/Green Deployments
 - AWS IAM
 - Database Activity Streams (DAS)
 - PostgreSQL logical replication
 - RDS Data API with Aurora PostgreSQL Serverless v2 and provisioned
 - RDS Proxy with RDS for SQL Server
 - Salted challenge response authentication mechanism (SCRAM)
 - Query Editor
 - Kerberos authentication via Active Directory